

Types of variables

1. *Var*
2. *Let*
3. *Const*

Var:

The **var** keyword in JavaScript is used to **declare a variable**. It's one of the older ways to declare variables (before *let* and *const* were **introduced in ES6**).

```
var variableName = value;
```

The **value** is optional — you can declare without initializing.

Key Characteristics of var:

1. Function-scoped

var is not **block-scoped**, It's scoped to the nearest function called **Function Scoped**, can **reassign** and **re-declare**

var - Default value = undefined

What does block-scoped mean?

A block in JavaScript is any code wrapped in **{ }** — such as **inside if, for, while, or even just plain { }**.

A block-scoped variable is **only accessible within that block and not outside of it**.

Example:01 [block-scoped]

```
{
  var z = 30;
  console.log(z); // 30
}

console.log(z); // ✔ 30 (still accessible outside the block!)
```

Example:02 [Function Scoped]

```
function testVar() {
  if (true) {
    var x = 10;
  }
  console.log(x); // 10 (accessible outside the block)
}
testVar();
```

Partially Hoisted vs Fully Hoisted

When JavaScript runs your code, it goes through two phases:

1. **Compilation Phase**: it "hoists" declarations to the top of their scope.
2. **Execution Phase** – it runs your code line-by-line.

Fully Hoisted = Declaration + Initialization

Function declarations are **fully hoisted**, so you can call them before they're written. For **function declarations**, JavaScript does both of these in the first phase:

- It hoists the function name
- It hoists the entire function body (the actual code inside)

So the function is ready to be used even before it appears in your code

```
sayHi(); // Works! Logs "Hi!"
function sayHi() {
  console.log("Hi!");
}
```

```
function sayHi() {
  console.log("Hi!");
}
sayHi(); // "Hi!"
```

Partially Hoisted = Declaration only (no value)

Applies to: **var**, **let**, **const**, **function expressions**, **arrow functions**.

Accessing it before the assignment can give you:

- `undefined` (for **var**)
- `ReferenceError` (for **let/const**)
- **let** is hoisted, but it's in the **Temporal Dead Zone (TCZ)**, so it can't be used yet.

Example with var : -----

```
console.log(x); // ✔ undefined
var x = 10;
```

Example with let / const : -----

```
console.log(y); // ✖ ReferenceError
let y = 20;
```

Example with function expression: -----

```
sayHi(); // ✖ TypeError: sayHi is not a function
var sayHi = function() {
  console.log("Hi!");
};
```

Temporal Dead Zone

in JavaScript, it's actually a super important concept when you're working with **let** and **const**.

The Temporal Dead Zone is the period between:

- The start of the scope (like a block or function), and
- The point where the variable is actually declared (initialized) in your code. Means **TDZ ends** once the line with the **let** or **const** declaration is evaluated.

During this time, the variable exists (because it was hoisted), but you can't access it — doing so throws a **ReferenceError**.

Why does this exist?

The TDZ helps catch bugs by preventing access to variables before they're safely defined.

Unlike `var`, which just gives you `undefined` if accessed early, `let` and `const` protect you.

Example with var : -----

```
console.log(a); // undefined – maybe you forgot to assign it?
var a = 10;
```

Example with let / const : -----

```
console.log(b); // ✖ ReferenceError – more obvious that something's wrong
let b = 10;
```

2. **Hoisting** [Hoisting is JavaScript's behavior of moving declarations to the top of their scope (before code is executed).]

Variables declared with `var` are **hoisted** to the top of their scope. But only the declaration, not the value.

```
console.log(a); // undefined (NOT an error!)
var a = 5;
```

let:

let is a keyword used to declare variables in JavaScript. It was introduced in ES6 (2015) to improve on some of the quirks of var.

Key Features of let

1. **Block-scoped:** Variables declared with let are only accessible within the block ({ }) they're defined in.

```
{
  let x = 10;
  console.log(x); // ✔ 10
}
console.log(x); // ✖ ReferenceError
```

2. **No redeclaration in the same scope:** You can't declare the same let variable twice in the same block. **But you can reassign:**

```
let name = 'Alice';
// let name = 'Bob'; // ✖ SyntaxError

let name = 'Alice';
name = 'Bob'; // ✔ allowed
```

3. **Hoisted but in the Temporal Dead Zone (TDZ):**

let is hoisted, but you can't use it before the line it's declared — doing so throws a ReferenceError.

```
console.log(a); // ✖ ReferenceError
let a = 5;
```

const:

const is used to declare variables that cannot be reassigned. It also came with ES6 (2015), like **let**

Example: const pi = 3.14159;

Key Features of const:

- Once you assign a value to a const, you can't change it.

```
const age = 30;
age = 31; // ✖ TypeError: Assignment to constant variable.
```

- **Block-scoped:** Like let, const is only accessible inside the block {} it's declared in.

```
{
  const x = 10;
  console.log(x); // ✔ 10
}
console.log(x); // ✖ ReferenceError
```

- **Hoisted but in Temporal Dead Zone:** Same as let — it's hoisted, but can't be accessed before the line it's declared on.

```
console.log(a); // ✖ ReferenceError
const a = 5;
```

- **const does NOT mean the value is immutable:** If the value is an object or array, the reference is constant, but the contents can change

```
// Object
const user = { name: 'Alice' };
user.name = 'Bob'; // ✔ Allowed
user = { name: 'Carol' } // ✖ Error: can't reassign the variable
// Array
const nums = [1, 2, 3];
nums.push(4); // ✔ Allowed
nums = [1, 2]; // ✖ Error
```

